

Università degli Studi

Dipartimento di Informatica

Corso di Programmazione Web

ParkShare

Una piattaforma web open-source
per la condivisione di parcheggi privati

Studente: Sandi Russo

Matricola: 553675

Docente: Andrea Nucita

A.A.: 2025/2026

Indice

1	Introduzione	3
1.1	Contesto e motivazione	3
1.2	Obiettivi del progetto	3
2	Stato dell'Arte	4
2.1	Il problema del parcheggio urbano	4
2.2	Soluzioni commerciali esistenti	4
2.2.1	Limitazioni delle soluzioni esistenti	4
2.3	Tecnologie cartografiche open-source	5
2.4	Servizi web: REST vs SOAP	5
3	Materiali e Metodi	6
3.1	Architettura del sistema	6
3.2	Scelte tecnologiche lato client	7
3.2.1	JavaScript Vanilla senza framework	7
3.2.2	Gestione asincrona con Fetch API	7
3.3	Scelte tecnologiche lato server	8
3.3.1	PHP senza framework	8
3.3.2	Pattern Repository	8
3.3.3	Singleton per la connessione al database	9
3.4	Il servizio web RESTful	9
3.4.1	Principi di progettazione	9
3.4.2	Formato delle risposte	10
3.5	Database relazionale	11
3.5.1	Schema Entità-Relazione	11
3.5.2	Ricerca geografica con la formula di Haversine	11
3.6	Componente cartografica: Leaflet.js e OpenStreetMap	12
3.6.1	Motivazioni della scelta	12
3.6.2	Reverse geocoding con Nominatim	12
3.7	Accessibilità e usabilità	13
3.7.1	Tema chiaro e scuro con persistenza	13
3.7.2	Accessibilità WCAG	13
3.7.3	Responsività mobile-first	14

3.8	Sicurezza	14
3.8.1	Gestione delle credenziali	14
3.8.2	Protezione contro attacchi comuni	14
3.9	Generazione PDF	15
4	Risultati	15
4.1	Funzionalità implementate	15
4.2	Interfacce utente	16
4.2.1	Pagina Home	16
4.2.2	Tema chiaro e scuro	17
4.3	Metriche del progetto	17
5	Conclusioni	17
5.1	Obiettivi raggiunti	18
5.2	Considerazioni critiche	18
5.3	Sviluppi futuri	18
5.4	Riflessioni finali	19

1 Introduzione

1.1 Contesto e motivazione

Nelle città moderne, il parcheggio rappresenta uno dei problemi logistici più ricorrenti per i guidatori: la ricerca di un posto libero genera traffico, perdita di tempo e frustrazione, con effetti negativi sia sull'esperienza dell'utente sia sull'ambiente. Parallelamente, molti proprietari di abitazioni o attività commerciali dispongono di posti auto privati che rimangono inutilizzati per la maggior parte della giornata.

L'idea alla base di **ParkShare** nasce proprio da questa asimmetria: collegare chi cerca un parcheggio con chi ne possiede uno disponibile, creando un *marketplace* di parcheggi privati accessibile via browser, senza installazione di applicazioni, senza costi di accesso per gli utenti e senza dipendenza da servizi commerciali a pagamento.

1.2 Obiettivi del progetto

Il progetto si propone di raggiungere i seguenti obiettivi:

1. **Funzionalità complete end-to-end:** registrazione utente, pubblicazione annunci, prenotazione con validazione oraria, sistema di recensioni, generazione di ricevute PDF.
2. **Indipendenza da API proprietarie:** tutta la componente cartografica è basata su tecnologie open-source (**Leaflet.js** + **OpenStreetMap**), senza richiedere chiavi API, senza costi di utilizzo e senza inviare dati degli utenti a terze parti.
3. **Accessibilità universale:** l'interfaccia supporta modalità chiara e scura (con persistenza della scelta), è completamente responsiva per dispositivi mobili, rispetta le linee guida WCAG per l'accessibilità (attributi ARIA, navigazione da tastiera) e onora la preferenza **prefers-reduced-motion** per gli utenti con sensibilità alle animazioni.
4. **Sicurezza applicativa:** gestione sicura delle credenziali tramite file `.env`, password cifrate con `bcrypt`, query parametrizzate contro SQL injection, rate limiting sul login.
5. **Qualità del codice:** architettura a strati con pattern Repository, separazione netta tra logica di business, accesso ai dati e presentazione.

2 Stato dell'Arte

2.1 Il problema del parcheggio urbano

Il problema della sosta urbana è ampiamente documentato in letteratura. Studi condotti in diverse città europee stimano che una quota compresa tra il 20% e il 30% del traffico cittadino sia generato da veicoli alla ricerca di parcheggio. In Italia, la situazione è aggravata dall'elevata densità veicolare nei centri storici, dalla scarsità di parcheggi pubblici e dalla presenza di numerosi posti auto privati non sfruttati in orari di punta.

2.2 Soluzioni commerciali esistenti

Negli ultimi anni sono emersi diversi servizi digitali per la gestione della sosta. I più noti a livello internazionale sono:

JustPark (UK) Piattaforma peer-to-peer per affitto di posti auto privati. Utilizza Google Maps API per la visualizzazione cartografica e addebita una commissione del 15% su ogni transazione.

SpotHero / ParkWhiz (USA) Aggregatori di parcheggi prevalentemente commerciali. Richiedono integrazione con sistemi di pagamento proprietari e sono geograficamente limitati al mercato nordamericano.

EasyPark Applicazione diffusa anche in Italia per il pagamento della sosta su strisce blu. Non gestisce parcheggi privati e richiede un account a pagamento per i gestori.

ParkU Piattaforma italiana di parking sharing, acquisita nel 2016 da EasyPark. Non più disponibile come servizio indipendente.

2.2.1 Limitazioni delle soluzioni esistenti

Tutte le soluzioni commerciali sopra citate presentano una o più delle seguenti criticità:

- **Dipendenza da API proprietarie a pagamento:** la quasi totalità utilizza Google Maps Platform, il cui piano gratuito è limitato a \$200/mese di credito.
- **Trasmissione di dati a terze parti:** l'utilizzo di Google Maps o Here Maps implica che le coordinate geografiche degli utenti vengano elaborate da server di proprietà di grandi aziende tecnologiche, con implicazioni per la privacy non sempre accettabili.

- **Lock-in tecnologico:** costruire un'applicazione su API commerciali significa legarsi a politiche di pricing che possono cambiare unilateralmente, come dimostrato dall'aumento dei prezzi di Google Maps nel 2018, che costrinse molti sviluppatori a migrare con urgenza.
- **Accessibilità limitata:** poche piattaforme supportano nativamente la modalità scura, la navigazione da tastiera o rispettano le preferenze di sistema per la riduzione del movimento.

2.3 Tecnologie cartografiche open-source

Di fronte alle limitazioni sopra descritte, il panorama open-source offre alternative mature e affidabili.

OpenStreetMap (OSM) è un progetto collaborativo avviato nel 2004 che mette a disposizione dati geografici liberamente utilizzabili sotto licenza ODbL. Con oltre 10 milioni di utenti contributori nel mondo, rappresenta il dataset geografico aperto più completo disponibile.

Leaflet.js è una libreria JavaScript open-source (licenza BSD-2) specializzata nella visualizzazione di mappe interattive. Con soli ~42 KB è significativamente più leggera delle controparti commerciali (Google Maps JS API supera i 300 KB), garantisce prestazioni elevate su dispositivi mobili e non richiede alcuna chiave API.

Nominatim è il servizio di geocoding (indirizzo → coordinate e viceversa) ufficiale di OpenStreetMap, utilizzato in ParkShare per la funzione di rilevamento automatico dell'indirizzo dal segnale GPS. Come Leaflet, è completamente gratuito e non trasmette dati identificativi a servizi terzi.

2.4 Servizi web: REST vs SOAP

La scelta tra le due principali architetture per i servizi web è caduta su **REST** (Representational State Transfer) per le seguenti ragioni:

- **Semplicità:** REST utilizza i metodi HTTP standard (GET, POST, PUT, DELETE) in modo semanticamente coerente con l'operazione da compiere, senza richiedere un livello di overhead aggiuntivo come l'envelope SOAP.
- **Formato dei dati:** JSON è notevolmente più compatto di XML e nativamente supportato da JavaScript, eliminando la necessità di parser dedicati.

3 Materiali e Metodi

3.1 Architettura del sistema

ParkShare segue un'architettura **client-server a tre livelli** (three-tier):

1. **Presentation tier** (client): browser web con HTML5, CSS3 e JavaScript vanilla, responsabile del rendering dell'interfaccia e della comunicazione con il tier successivo tramite chiamate **fetch** asincrone.
2. **Application tier** (server): PHP 8 che espone endpoint RESTful, esegue la logica di business, valida gli input e gestisce la sessione utente.
3. **Data tier**: database MySQL con schema relazionale, accessibile esclusivamente tramite lo strato applicativo.

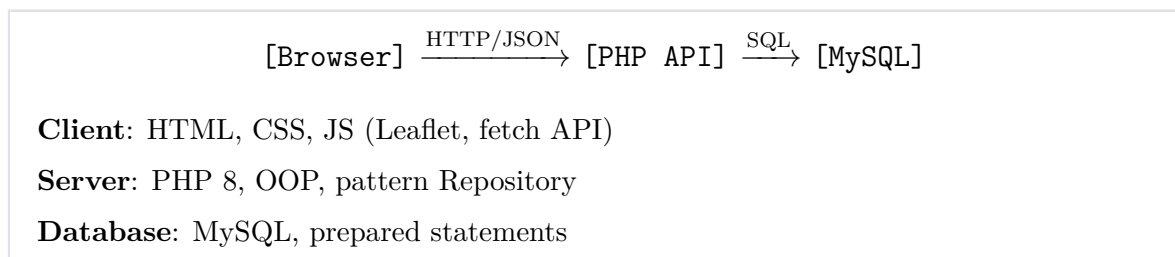


Figura 1: Schema dell'architettura a tre livelli di ParkShare.

La struttura delle directory riflette questa separazione:

```

1 PROGRAMMAZIONE_WEB/
2 |- api/
3 |- classes/
4 |- config/
5 |- includes/
6 |- pages/
7 |- assets/
8 |   |- css/
9 |   \- js/
10 |- .env
11 \- .env.example

```

Listing 1: Struttura del progetto ParkShare.

3.2 Scelte tecnologiche lato client

3.2.1 JavaScript Vanilla senza framework

Una scelta progettuale deliberata è stata quella di **non adottare framework JavaScript** come React, Vue o Angular. Le motivazioni sono molteplici:

- **Comprensione dei fondamentali:** un progetto universitario ha lo scopo primario di dimostrare la padronanza delle tecnologie di base. Delegare la gestione del DOM, degli eventi e delle chiamate asincrone a un framework nasconde i meccanismi sottostanti.
- **Zero dipendenze da npm:** nessun `node_modules`, nessun bundler (webpack, Vite), nessun processo di build. Il progetto si esegue servendo i file statici da un qualsiasi server PHP, senza configurazioni aggiuntive.
- **Longevità:** il codice scritto con API Web native (Fetch API, DOM API, ES2020+) non diventa obsoleto con i cicli di rilascio dei framework.

3.2.2 Gestione asincrona con Fetch API

Tutta la comunicazione con il server avviene tramite la **Fetch API** con la sintassi `async/await`, che rende il flusso asincrono leggibile quanto del codice sincrono. Le funzioni helper `postJSON`, `putJSON` e `deleteJSON` centralizzano la gestione degli errori di rete:

```
1 async function postJSON(url, body) {  
2     try {  
3         const res = await fetch(url, {  
4             method: 'POST',  
5             headers: { 'Content-Type': 'application/json' },  
6             body: JSON.stringify(body)  
7         });  
8         return await res.json();  
9     } catch {  
10        return { success: false,  
11            error: 'Errore di rete. Controlla la  
12                connessione.' };  
13    }  
}
```


Listing 2: Funzione helper per richieste POST in JSON.

3.3 Scelte tecnologiche lato server

3.3.1 PHP senza framework

Analogamente al lato client, si è scelto di non adottare framework PHP come Laravel o Symfony. Questo permette di:

- Dimostrare la comprensione diretta di PHP: routing manuale, gestione delle sessioni, preparazione delle query, upload di file.
- Mantenere un'installazione minimale: l'unica dipendenza esterna è **dompdf** (via Composer) per la generazione dei PDF.
- Controllare ogni aspetto della sicurezza senza fare affidamento “cieco” sulle protezioni automatiche di un framework.

3.3.2 Pattern Repository

La logica di accesso al database è incapsulata nel pattern **Repository**, implementato tramite una classe astratta base:

```
1 abstract class Repository {  
2     protected static function fetchOne(  
3         string $sql, string $types, mixed ...$params  
4     ): ?array { ... }  
5  
6     protected static function fetchAll(  
7         string $sql, string $types, mixed ...$params  
8     ): array { ... }  
9 }
```

Listing 3: Classe Repository — metodi di accesso generici al DB.

Le classi di dominio (**User**, **Parcheggio**, **Prenotazione**, **Recensione**) estendono **Repository** e implementano la logica specifica. Questo garantisce che ogni query al database passi obbligatoriamente per i prepared statements, eliminando alla radice il rischio di SQL injection.

3.3.3 Singleton per la connessione al database

La classe `Database` implementa il pattern **Singleton** per garantire che esista una sola connessione MySQLi per richiesta HTTP, evitando lo spreco di risorse derivante dall'apertura di connessioni multiple:

```
1 class Database {
2     private static ?Database $instance = null;
3     private mysqli $connection;
4
5     private function __construct() {
6         $this->connection = new mysqli(
7             DB_HOST, DB_USER, DB_PASS, DB_NAME
8         );
9     }
10
11     public static function getInstance(): Database {
12         if (self::$instance === null) {
13             self::$instance = new Database();
14         }
15         return self::$instance;
16     }
17 }
```

Listing 4: Pattern Singleton per la connessione al database.

3.4 Il servizio web RESTful

3.4.1 Principi di progettazione

L'API di ParkShare rispetta i principi REST di Roy Fielding: risorse identificate da URI, operazioni espresse tramite metodi HTTP semantici, comunicazione stateless, risposte in formato JSON.

Tabella 1: Endpoint principali dell'API RESTful di ParkShare.

Metodo	Endpoint	Azione	Auth?
POST	/api/auth.php (login)	Autenticazione utente	No
POST	/api/auth.php (register)	Registrazione utente	No
POST	/api/auth.php (logout)	Terminazione sessione	Sì
GET	/api/parcheggi.php?id=N	Dettaglio parcheggio	No
GET	/api/parcheggi.php?pagina=N	Lista paginata	No
GET	/api/parcheggi.php?lat=&lon=	Ricerca per posizione	No
POST	/api/parcheggi.php	Crea parcheggio	Sì
PUT	/api/parcheggi.php	Modifica parcheggio	Sì
DELETE	/api/parcheggi.php	Elimina parcheggio	Sì
POST	/api/prenotazioni.php	Crea prenotazione	Sì
PUT	/api/prenotazioni.php	Aggiorna stato prenotazione	Sì
GET	/api/recensioni.php	Recensioni di un parcheggio	No
POST	/api/recensioni.php	Crea recensione	Sì
DELETE	/api/recensioni.php	Elimina recensione	Sì
POST	/api/utenti.php	Aggiorna profilo utente	Sì
GET	/api/ricevuta.php?id=N	Genera ricevuta PDF	Sì

3.4.2 Formato delle risposte

Tutte le risposte adottano una struttura JSON uniforme:

```

1 // Risposta di successo
2 { "success": true, "data": { ... } }
3
4 // Risposta di errore
5 { "error": "Messaggio di errore leggibile dall'utente" }
6
7 // Risposta paginata (endpoint /api/parcheggi.php?pagina=N)

```

```

8 {
9   "success": true,
10  "data": [ ... ],
11  "totale": 45,
12  "pagina": 2,
13  "perPagina": 12,
14  "hasMore": true
15 }

```

Listing 5: Struttura delle risposte API — successo ed errore.

3.5 Database relazionale

3.5.1 Schema Entità-Relazione

Il database è composto da cinque tabelle principali, con relazioni che garantiscono l'integrità referenziale:

- **UTENTE**: ID_UTENTE, NOME, COGNOME, EMAIL (unique), PASSWORD_HASH.
- **PARCHEGGIO**: ID_PARCHEGGIO, ID_PROPRIETARIO (FK → UTENTE), TITOLO, DESCRIZIONE, INDIRIZZO, LAT, LON, COSTO_ORARIO, ORA_APERTURA, ORA_CHIUSURA, FOTO.
- **DISPONIBILITA**: ID_PARCHEGGIO (FK), GIORNO (0=Lun ... 6=Dom). Relazione N:M tra parcheggio e giorni della settimana.
- **PRENOTAZIONE**: ID_PRENOTAZIONE, ID_UTENTE (FK), ID_PARCHEGGIO (FK), DATA, ORA_INIZIO, ORA_FINE, IMPORTO_TOTALE, STATO (in_attesa | confermata | cancellata).
- **RECENSIONE**: ID_RECENSIONE, ID_UTENTE (FK), ID_PARCHEGGIO (FK), ID_PRENOTAZIONE (FK, unique — una sola recensione per prenotazione), VOTO (1–5), COMMENTO, DATA.

3.5.2 Ricerca geografica con la formula di Haversine

La funzionalità “cerca vicino a me” utilizza la **formula di Haversine** direttamente in una query MySQL per calcolare la distanza in chilometri tra le coordinate GPS dell'utente e quelle di ogni parcheggio:

```

1 SELECT *,
2   (6371 * ACOS (
3     COS (RADIANS (?) ) * COS (RADIANS (LAT)) *

```

```
4     COS(RADIANS(LON) - RADIANS(?)) +  
5     SIN(RADIANS(?)) * SIN(RADIANS(LAT))  
6 )) AS distanza  
7 FROM PARCHEGGIO  
8 WHERE LAT BETWEEN ? AND ?  
9     AND LON BETWEEN ? AND ?  
10 HAVING distanza <= ?  
11 ORDER BY distanza ASC
```

Listing 6: Ricerca geografica con Haversine e bounding box.

Il bounding box approssimativo riduce il numero di righe su cui calcolare la funzione trigonometrica, migliorando le prestazioni al crescere del numero di parcheggi.

3.6 Componente cartografica: Leaflet.js e OpenStreetMap

3.6.1 Motivazioni della scelta

La decisione di non utilizzare Google Maps API e di affidarsi invece a **Leaflet.js** con tile di **OpenStreetMap** è stata motivata da tre ragioni fondamentali:

1. **Costo zero e nessuna chiave API:** il progetto funziona immediatamente senza registrazioni, senza limiti di utilizzo mensile e senza rischi di blocco per superamento della quota gratuita.
2. **Privacy degli utenti:** le richieste di tile vengono inviate ai server OSM (o a mirror della community come `tile.openstreetmap.org`), non ai server di Google. Nessun dato di navigazione viene trasmesso a terze parti commerciali.
3. **Leggerezza:** Leaflet pesa ~42 KB gzippati contro i >300 KB di Google Maps JS API v3, con impatto diretto sui tempi di caricamento su connessioni lente.

3.6.2 Reverse geocoding con Nominatim

Per la compilazione automatica dell'indirizzo durante l'aggiunta di un parcheggio, si utilizza il servizio **Nominatim** di OpenStreetMap, che converte le coordinate GPS in un indirizzo leggibile:

```
1 const res = await fetch(  
2     'https://nominatim.openstreetmap.org/reverse' +  
3     '?lat=${lat}&lon=${lon}&format=json',
```

```
4     { headers: { 'Accept-Language': 'it' } }  
5 );  
6 const dati = await res.json();  
7 const indirizzo = `${dati.address.road}, ${dati.address.city}`;
```

Listing 7: Chiamata a Nominatim per il reverse geocoding.

3.7 Accessibilità e usabilità

3.7.1 Tema chiaro e scuro con persistenza

ParkShare supporta nativamente tre modalità di visualizzazione:

1. **Automatica:** rileva la preferenza di sistema tramite la media query CSS `prefers-color-scheme` e imposta il tema corrispondente.
2. **Scuro manuale:** l'utente attiva il tema scuro tramite il bottone in navbar; la scelta viene salvata in `localStorage` e persiste tra sessioni diverse.
3. **Chiara manuale:** forza il tema chiaro anche su sistemi con preferenza di sistema impostata su scuro.

L'implementazione sfrutta variabili CSS custom (`-bg`, `-surface`, `-text`, ecc.) e l'attributo `data-theme` sull'elemento `<html>`, evitando la duplicazione di stili e rendendo i cambi di tema istantanei senza ricaricare la pagina.

3.7.2 Accessibilità WCAG

L'interfaccia è progettata per essere utilizzabile da tutti:

- **Attributi ARIA:** i dialog (modal login, modal recensione) usano `role="dialog"`, `aria-modal="true"` e `aria-labelledby` per una corretta lettura da parte degli screen reader.
- **Navigazione da tastiera:** le stelle di valutazione hanno `role="button"`, `tabindex="0"` e gestiscono `keydown` (Enter/Spazio), permettendo il voto senza mouse.
- **Skip-to-content:** un link nascosto “Salta al contenuto” consente agli utenti di tastiera di bypassare la navbar. Utilizza `:focus-visible` invece di `:focus` per non apparire al tocco su dispositivi mobili.

- **Riduzione del movimento:** la media query `prefers-reduced-motion: reduce` disabilita tutte le animazioni CSS e JavaScript per gli utenti che ne fanno richiesta.
- **Stili di stampa:** la media query `@media print` nasconde navbar, bottoni e filtri, producendo ricevute stampabili pulite.

3.7.3 Responsività mobile-first

Il CSS è scritto con approccio **mobile-first**: gli stili base sono ottimizzati per schermi piccoli, con override progressivi al crescere della risoluzione. Su mobile compare una bottom navigation bar fissa con link rapidi a Home, Aggiungi parcheggio e Profilo. Le griglie di card utilizzano CSS Grid con `auto-fit` e `minmax()` per adattarsi fluidamente a qualsiasi larghezza senza breakpoint rigidi.

3.8 Sicurezza

3.8.1 Gestione delle credenziali

Le credenziali del database non sono mai presenti nel codice sorgente. Un parser leggero incluso in `config.php` legge le variabili da un file `.env` escluso dal controllo di versione tramite `.gitignore`:

```
1 $envFile = dirname(__DIR__) . '/.env';
2 if (file_exists($envFile)) {
3     foreach (file($envFile, FILE_IGNORE_NEW_LINES) as $line) {
4         if (str_starts_with(trim($line), '#')) continue;
5         [$k, $v] = array_map('trim', explode('=', $line, 2));
6         if (!empty($k)) putenv("$k=$v");
7     }
8 }
9 define('DB_PASS', getenv('DB_PASS') ?: '');
```

Listing 8: Caricamento sicuro delle variabili d'ambiente.

3.8.2 Protezione contro attacchi comuni

- **SQL Injection:** ogni query al database utilizza prepared statements MySQLi con `bind_param`, rendendo impossibile l'iniezione di codice SQL attraverso gli input utente.

- **Password sicure:** le password vengono cifrate con `password_hash()` (algoritmo bcrypt, fattore di costo 12) e verificate con `password_verify()`.
- **Rate limiting:** l'endpoint di login traccia i tentativi falliti nella sessione PHP e blocca ulteriori tentativi per 5 minuti dopo 5 errori consecutivi.
- **Validazione upload:** i file immagine caricati vengono verificati tramite la classe `finfo` (rilevamento MIME reale, non basato sull'estensione del file) e limitati a 5 MB.
- **XSS:** tutti i dati provenienti dal database vengono passati attraverso `htmlspecialchars()` prima di essere inseriti nel DOM lato PHP; lato JavaScript si usano le proprietà `textContent` per i valori dinamici.

3.9 Generazione PDF

Le ricevute di prenotazione vengono generate lato server con la libreria open-source **dompdf**, che converte HTML/CSS in PDF senza dipendere da servizi cloud. La ricevuta include: dati del cliente, dati del proprietario del parcheggio, dettagli della prenotazione (data, orario, importo totale) e badge dello stato. La funzione `date_default_timezone_set('Europe/Rome')` in `config.php` garantisce che il timestamp di generazione rifletta il fuso orario italiano indipendentemente dalla configurazione del server.

4 Risultati

4.1 Funzionalità implementate

L'applicazione implementa un ciclo completo di utilizzo, dal primo accesso alla generazione della ricevuta:

1. **Registrazione e autenticazione:** form di registrazione con validazione lato client (formato email, lunghezza password minima 8 caratteri) e lato server (unicità email, cifratura bcrypt). Il login è protetto da rate limiting (max 5 tentativi per 5 minuti).
2. **Home con filtri avanzati:** griglia di parcheggi con filtri combinabili in tempo reale (testo libero, prezzo massimo, valutazione minima, solo aperti oggi, ordinamento). I filtri operano sul DOM senza chiamate server aggiuntive. Il pulsante “Carica altri” carica i parcheggi successivi tramite AJAX mantenendo i filtri attivi.

3. **Ricerca geografica:** il pulsante “Cerca vicino a me” ottiene le coordinate GPS tramite `navigator.geolocation` e interroga l’API con la formula di Haversine, restituendo i parcheggi entro un raggio configurabile (default 5 km).
4. **Dettaglio parcheggio:** pagina con mappa interattiva Leaflet, orari, disponibilità per giorno della settimana, form di prenotazione con calcolo automatico dell’importo, sezione recensioni con media e voti.
5. **Prenotazione:** validazione del giorno selezionato rispetto alla disponibilità del parcheggio sia lato client che server; controllo della disponibilità oraria (nessuna sovrapposizione con prenotazioni esistenti); transizione automatica di stato `in_attesa` → `confermata` → `completata` (quest’ultima attivata al primo accesso al profilo dopo la data di scadenza).
6. **Recensioni:** gli utenti possono lasciare una recensione (1–5 stelle + commento opzionale) per ogni prenotazione completata, sia dalla pagina del proprio profilo sia direttamente dalla pagina del parcheggio. È consentita una sola recensione per prenotazione (vincolo unico in database).
7. **Profilo utente:** riepilogo prenotazioni con badge di stato, azioni per cancellare una prenotazione attiva, download della ricevuta PDF, gestione dei propri parcheggi pubblicati con possibilità di eliminazione.
8. **Aggiunta parcheggio:** form con compilazione automatica dell’indirizzo via GPS (Nominatim), selezione dei giorni disponibili, orari di apertura/chiusura, upload foto con validazione MIME e dimensione massima (5 MB).
9. **Ricevuta PDF:** documento generato lato server con `dompdf` contenente dati del cliente, dati del proprietario, dettaglio della prenotazione e importo totale. Include stili di stampa ottimizzati (`@media print`).

4.2 Interfacce utente

4.2.1 Pagina Home

La home presenta la griglia di parcheggi con:

- Barra di ricerca testuale in navbar (funziona su tutte le pagine con redirect automatico alla home se non già presenti).

- Pannello filtri avanzati (collassabile, con pulsante toggle e attributo `aria-expanded` aggiornato dinamicamente).
- Badge “Aperto oggi” calcolato in PHP sulla base del giorno corrente.
- Contatore “Mostrati X di Y parcheggi” aggiornato dopo ogni “Carica altri”.

4.2.2 Tema chiaro e scuro

Tabella 2: Variabili CSS principali nei due temi.

Variabile	Tema chiaro	Tema scuro
<code>--bg</code>	<code>#ffffff</code>	<code>#0f172a</code>
<code>--surface</code>	<code>#f7f7f7</code>	<code>#1e293b</code>
<code>--accent</code>	<code>#185FA5</code>	<code>#3b82f6</code>
<code>--text</code>	<code>#1a1a1a</code>	<code>#f1f5f9</code>
<code>--muted</code>	<code>#6b6b6b</code>	<code>#94a3b8</code>
<code>--border</code>	<code>rgba(0,0,0,.1)</code>	<code>rgba(255,255,255,.1)</code>

4.3 Metriche del progetto

Tabella 3: Dimensione dei componenti principali del progetto.

File	Righe	Note
<code>assets/js/main.js</code>	~780	JS client-side (no dipendenze)
<code>assets/css/style.css</code>	~1,450	CSS con variabili, dark mode, print
<code>classes/*.php</code>	~800	6 classi PHP con PHPDoc
<code>api/*.php</code>	~450	6 endpoint RESTful
<code>pages/*.php</code>	~500	5 viste PHP
Totale	~3,980	

5 Conclusioni

5.1 Obiettivi raggiunti

Il progetto ParkShare ha raggiunto tutti gli obiettivi prefissati:

- È stata realizzata un'applicazione web completa che copre l'intero ciclo di utilizzo: dalla scoperta di un parcheggio alla prenotazione, dalla recensione alla ricevuta PDF.
- L'architettura RESTful è implementata con rispetto dei principi fondamentali (metodi HTTP semantici, statelessness, formato JSON uniforme).
- La componente cartografica è completamente open-source e gratuita, senza dipendenza da Google Maps o altri servizi commerciali.
- L'interfaccia è accessibile, responsiva e usabile su qualsiasi dispositivo e con qualsiasi preferenza di sistema (tema chiaro/scuro, riduzione del movimento).
- Il codice adotta pratiche di sicurezza consolidate: prepared statements, bcrypt, rate limiting, gestione sicura delle credenziali tramite `.env`.

5.2 Considerazioni critiche

Alcune limitazioni del progetto attuale meritano di essere riconosciute:

- **Pagamenti:** il sistema attuale non integra un gateway di pagamento reale. Le prenotazioni vengono confermate automaticamente senza transazione economica, il che nella realtà richiederebbe l'integrazione con un servizio come Stripe o PayPal.
- **CSRF:** la protezione contro il Cross-Site Request Forgery non è stata implementata nella versione corrente. In produzione, ogni endpoint che modifica dati dovrebbe validare un token CSRF.
- **Notifiche:** non è previsto un sistema di notifica (email o push) per eventi come la conferma di una prenotazione o la ricezione di una recensione.

5.3 Sviluppi futuri

Gli sviluppi più interessanti per una versione futura del progetto sarebbero:

1. **Integrazione pagamenti:** connessione con Stripe Checkout per gestire transazioni reali, con ricevuta fiscale automatica.

2. **Notifiche email:** invio di email transazionali (conferma prenotazione, promemoria) tramite SMTP, senza dipendere da servizi a pagamento grazie a librerie come PHPMailer con server SMTP gratuiti.
3. **Progressive Web App (PWA):** aggiunta di un Service Worker per permettere l'installazione dell'applicazione come app nativa e il funzionamento offline di base.
4. **Pannello statistiche proprietario:** dashboard per i proprietari dei parcheggi con grafici di prenotazioni, guadagni e valutazioni nel tempo.
5. **CSRF protection:** implementazione di token sincronizzati per la protezione completa contro attacchi Cross-Site Request Forgery.

5.4 Riflessioni finali

Lo sviluppo di ParkShare ha rappresentato un'occasione per applicare concretamente le tecnologie studiate nel corso, affrontando problemi reali: come bilanciare usabilità e prestazioni, come strutturare un'API RESTful coerente, come proteggere un'applicazione dagli attacchi più comuni, come garantire l'accessibilità senza sacrificare l'estetica.

La scelta consapevole di evitare framework e librerie pesanti — sia lato client che lato server — ha reso il progetto più didatticamente significativo: ogni meccanismo è implementato in modo esplicito e comprensibile, senza “magia nera” nascosta nelle dipendenze.

L'adozione di tecnologie open-source per la componente cartografica (Leaflet + OpenStreetMap) dimostra come sia possibile realizzare applicazioni professionali senza costi di licensing e senza compromettere la privacy degli utenti — un principio che diventa sempre più rilevante nel panorama digitale contemporaneo.